

Checkpoint 6 — Data Infrastructure & Batch Profile Computation

Yassine — Amen Bank Internship
Direction Centrale du Système d'Information

June 16, 2026

Contents

1	Session Objectives	2
2	Generator Updates	2
2.1	New CSV Exports	2
2.2	Branch-Linked Employee Pool	2
2.3	New Client Fields	2
3	PostgreSQL Cold Store	3
3.1	Database Schema	3
3.2	Indexes	3
3.3	Data Loading	3
4	Batch Profile Computation	3
4.1	Architecture	3
4.2	Profile Structure — 122 Fields	3
4.3	Category 2 — Transaction Stats (96 fields)	4
4.4	Category 3 — Behavioral Patterns (14 fields)	4
4.5	Category 4 — Financial State (6 fields)	4
4.6	Category 6 — Recent Events Buffer	4
4.7	Sample Profile Validation	4
5	Generator Gaps Status	5
6	Current Pipeline Status	5
7	Next Session Plan	5

1 Session Objectives

This session transitioned the project from the data generation phase into the data infrastructure phase. The goals were:

1. Extend the generator to export master table CSVs alongside the existing transactions CSV.
2. Stand up the PostgreSQL cold store schema and load all generated data.
3. Implement batch profile computation, producing client profiles stored as JSONB snapshots.

All three objectives were completed.

2 Generator Updates

2.1 New CSV Exports

The generator now produces four CSV files per run:

File	Target Table	Row Count
transactions.csv	transactions	340,839
clients_master.csv	clients_master	10,000
accounts.csv	accounts	10,000
employees_master.csv	employees_master	300

2.2 Branch-Linked Employee Pool

Previously, employee IDs were assigned randomly to transactions from a flat global pool. This made the dual-actor model's employee peer comparison features (z-scores for volume, client concentration, location mismatch) meaningless.

The employee pool is now structured as a dictionary mapping each branch to its assigned tellers (~2 per branch). When generating a transaction, the employee is selected from the branch the client visits, ensuring realistic branch–employee linkage. Cross-check validation confirmed 0 mismatches between transaction-level and master-level branch assignments.

2.3 New Client Fields

Three fields were added to the `Client` class:

Field	Source	Purpose
account_opening_date	Archetype-driven sampling	Maturity status / cold-start
client_type	Static mapping from archetype	Regulatory classification
maturity_status	Derived (>90 days = mature)	Cold-start defense layer

Account age ranges are calibrated by archetype to ensure realism:

Archetype	Min Years	Max Years
Student	1	3
Small Business	1	10
Salaried	2	15
Big Business	5	20

3 PostgreSQL Cold Store

3.1 Database Schema

Database `amen_anomaly` was created on PostgreSQL 17 (localhost) with the following tables:

Table	Primary Key	Foreign Keys
<code>employees_master</code>	<code>employee_id</code>	—
<code>clients_master</code>	<code>client_id</code> (UUID)	—
<code>accounts</code>	<code>account_id</code> (UUID)	<code>client_id</code> → <code>clients_master</code>
<code>transactions</code>	<code>event_id</code> (UUID)	<code>client_id</code> , <code>account_id</code> , <code>employee_id</code>
<code>profile_snapshots</code>	<code>(client_id, computed_at)</code>	<code>client_id</code> → <code>clients_master</code>

3.2 Indexes

Two indexes were created to accelerate batch profile computation queries:

- `idx_tx_client_time` on `transactions(client_id, timestamp)`
- `idx_tx_employee_time` on `transactions(employee_id, timestamp)`

3.3 Data Loading

Data was loaded via `psycpg2.copy_expert` using PostgreSQL’s `COPY` protocol for bulk performance. Loading order respects the foreign key chain: `employees` → `clients` → `accounts` → `transactions`.

4 Batch Profile Computation

4.1 Architecture

The batch profile computation reads from the `transactions` table, computes per-client aggregate features, and stores the result as a JSONB blob in `profile_snapshots`. The reference date for window calculations is the last day of the simulation (2025-06-29). In production, this would be `datetime.now()`, executed nightly.

The JSONB storage design was chosen over individual columns for three reasons: schema flexibility as features evolve, direct serialization to Redis when the hot-path is wired up, and avoidance of DDL migrations on feature changes.

4.2 Profile Structure — 122 Fields

Category	Description	Fields	Status
1. Personal Info	Archetype, client type, branch, age, maturity	5	Done
2. Transaction Stats	Per operation × window (count/sum/mean/std/min/max)	96	Done
3. Behavioral Patterns	5 distributions × 2 windows + counterparties	14	Done
4. Financial State	Inflow/outflow/net at 30d and 90d	6	Done
5. Risk History	Per alert tier × window	0	Skipped*
6. Events Buffer	Last 50 events for LSTM input	1	Done
Total		122	

*Risk history requires the alerts table to be populated, which depends on the Decision Service. This category will be added when the feedback loop is implemented.

4.3 Category 2 — Transaction Stats (96 fields)

For each combination of operation type and time window, six statistics are computed on the `amount` column:

- **Operations:** retrait, versement, virement, cheque (4)
- **Windows:** 24h, 7d, 30d, 90d (4)
- **Statistics:** count, sum, mean, std, min, max (6)

Naming convention: `tx_{op}_{window}_{stat}`, e.g., `tx_retrait_30d_mean`.

4.4 Category 3 — Behavioral Patterns (14 fields)

Five probability distributions computed at 30d and 180d windows:

Distribution	What It Captures
<code>operation</code>	Proportion of each operation type
<code>hour</code>	Proportion of transactions per hour of day
<code>day_of_week</code>	Proportion per weekday
<code>branch</code>	Proportion per branch visited
<code>amount_buckets</code>	Proportion per amount range (7 bins)

The dual-window design (30d vs 180d) enables behavioral drift detection: the Scoring Service will compute Jensen–Shannon divergence between the two windows to quantify how much recent behavior deviates from the long-term baseline.

Additionally, four counterparty fields are computed over a 365-day window: `known_beneficiaries_365d` (set), `known_beneficiaries_count`, `known_emitters_365d` (set), `known_emitters_count`.

4.5 Category 4 — Financial State (6 fields)

Cumulative flow proxies computed at 30d and 90d windows:

- **Inflow:** sum of versement + cheque amounts
- **Outflow:** sum of retrait + virement amounts
- **Net:** inflow – outflow

4.6 Category 6 — Recent Events Buffer

The last 50 transactions for each client, ordered chronologically, stored as a JSON array. Each entry contains: `timestamp`, `operation_type`, `amount`, `branch_id`, `employee_id`. This buffer is the direct input to the LSTM at inference time. The buffer length of 50 matches the locked LSTM training sequence length to prevent context mismatch.

4.7 Sample Profile Validation

Spot-check of a salaried client (account age 1,186 days):

- Branch distribution: 83% home branch (BR-021) — consistent with ~0.90 archetype loyalty
- Operation mix: 33% retrait, 67% virement — close to salaried archetype (50/40 split)
- Amount range: 67% in 500–1,000 DT, 33% in 100–500 DT — realistic for salaried amounts
- Transaction hours: spread across 8h–14h — business hours only (guichet channel)

5 Generator Gaps Status

Gap	Status	Impact
Random employee assignment	Resolved	Branch-linked pool
Missing <code>account_opening_date</code>	Resolved	Archetype-driven
Missing master table exports	Resolved	3 new CSV exports
Stub anomaly scenarios	Parked	Less realistic anomalies
Placeholder counterparties	Parked	No real beneficiary graph
Known-beneficiary tracking	Parked	Counterparties are stubs

Parked gaps produce less realistic data but do not block the downstream pipeline. They will be revisited only if model evaluation indicates the data quality is insufficient.

6 Current Pipeline Status

Component	Status
Source Extraction (S1–S3)	Complete
Schema Design (Checkpoints 1–4)	Complete
Synthetic Data Generator (Checkpoint 5)	Complete
PostgreSQL Cold Store	Complete (this session)
Batch Profile Computation	Complete (this session)
Redis Hot Store	Not started
Scoring Service (Autoencoder + LSTM)	Next session
Streaming Services (Enrichment/Scoring/Decision)	Not started
Dashboard (Streamlit)	Not started
Monitoring (Prometheus/Grafana)	Not started
MLOps (MLflow)	Not started

7 Next Session Plan

The next session will focus on the Scoring Service:

1. Define the input feature vector — which of the 122 profile fields feed the models.
2. Prepare training data from `profile_snapshots`.
3. Design the Autoencoder architecture (reconstruction-based anomaly scoring).
4. Design the LSTM architecture (sequence prediction with 6 multi-task heads).
5. Implement the training pipeline with MLflow experiment tracking.